

R Lab Week 4 Part 1

Robert Gulotty

9/30/2016

```
library(stringr)
library(dplyr, warn.conflicts = FALSE)
library(ggplot2)
library(readr)
library(tidyr)
```

The TIDYVERSE: How to clean messy data

Much of the time spend in quantitative analysis is not exploring data, deriving estimators, or in writing up results. Instead, it is invariably spent cleaning data. Consider the following data downloaded from the World Bank.

```
WDI <- read_csv("~/Dropbox/mathCamp/Code/data/GlobalTradePctGDP19602016.csv")
```

```
## Parsed with column specification:
## cols(
##   .default = col_double(),
##   `Country Name` = col_character(),
##   `Country Code` = col_character(),
##   `Indicator Name` = col_character(),
##   `Indicator Code` = col_character(),
##   `2016` = col_character()
## )
## See spec(...) for full column specifications.
```

```
WDI %>% select(`Country Name`, `Indicator Name`, `1960`:`2016`)
```

```
## # A tibble: 264 × 59
##       `Country Name` `Indicator Name`   `1960`   `1961`   `1962`
##           <chr>         <chr>      <dbl>    <dbl>    <dbl>
## 1      Aruba Trade (% of GDP)      NA      NA      NA
## 2    Andorra Trade (% of GDP)      NA      NA      NA
## 3 Afghanistan Trade (% of GDP) 11.15703 12.55061 14.22764
## 4      Angola Trade (% of GDP)      NA      NA      NA
## 5     Albania Trade (% of GDP)      NA      NA      NA
## 6   Arab World Trade (% of GDP)      NA      NA      NA
## 7 United Arab Emirates Trade (% of GDP)      NA      NA      NA
## 8     Argentina Trade (% of GDP) 15.20810 11.98989 14.07553
## 9      Armenia Trade (% of GDP)      NA      NA      NA
## 10 American Samoa Trade (% of GDP)      NA      NA      NA
## # ... with 254 more rows, and 54 more variables: `1963` <dbl>,
## #   `1964` <dbl>, `1965` <dbl>, `1966` <dbl>, `1967` <dbl>, `1968` <dbl>,
## #   `1969` <dbl>, `1970` <dbl>, `1971` <dbl>, `1972` <dbl>, `1973` <dbl>,
## #   `1974` <dbl>, `1975` <dbl>, `1976` <dbl>, `1977` <dbl>, `1978` <dbl>,
## #   `1979` <dbl>, `1980` <dbl>, `1981` <dbl>, `1982` <dbl>, `1983` <dbl>,
```

```
## # `1984` <dbl>, `1985` <dbl>, `1986` <dbl>, `1987` <dbl>, `1988` <dbl>,
## # `1989` <dbl>, `1990` <dbl>, `1991` <dbl>, `1992` <dbl>, `1993` <dbl>,
## # `1994` <dbl>, `1995` <dbl>, `1996` <dbl>, `1997` <dbl>, `1998` <dbl>,
## # `1999` <dbl>, `2000` <dbl>, `2001` <dbl>, `2002` <dbl>, `2003` <dbl>,
## # `2004` <dbl>, `2005` <dbl>, `2006` <dbl>, `2007` <dbl>, `2008` <dbl>,
## # `2009` <dbl>, `2010` <dbl>, `2011` <dbl>, `2012` <dbl>, `2013` <dbl>,
## # `2014` <dbl>, `2015` <dbl>, `2016` <chr>
```

Notice that we use the `read_csv` function, rather than the base R function. This function is much faster, and is explicit about how it treats the columns, so you never make a mistake on that front.

These data contain an indicator for the level of interdependence experienced by nations, imports plus exports divided by GDP for countries from 1960-2015. The data is in an untidy format, with years along the top, and variable names in each of the rows. *tidy data* is data that has exactly one observation per row, with columns for each variable. Here each row contains up to 57 observations.

Gather the columns

The `tidyr` package has a solution which gathers all these columns into one. You identify a *key*, in this case the Year, which will contain the variable names from the original dataset. You then give the name for the new column which will be filled in for each _____value_____ of the original dataset. Finally, you list all the row characteristics you do not want to be gathered.

```
WDI %>% select(`Country Name`, `1960`:`2016`) %>% gather(key = Year,
  value = Trade, -`Country Name`)
```

```
## # A tibble: 15,048 × 3
##       `Country Name`   Year      Trade
##       <chr> <chr>    <chr>
## 1      Aruba  1960    <NA>
## 2    Andorra  1960    <NA>
## 3 Afghanistan 1960 11.15702673
## 4      Angola  1960    <NA>
## 5     Albania  1960    <NA>
## 6   Arab World  1960    <NA>
## 7 United Arab Emirates 1960    <NA>
## 8     Argentina 1960 15.20809609
## 9      Armenia  1960    <NA>
## 10 American Samoa 1960    <NA>
## # ... with 15,038 more rows
```

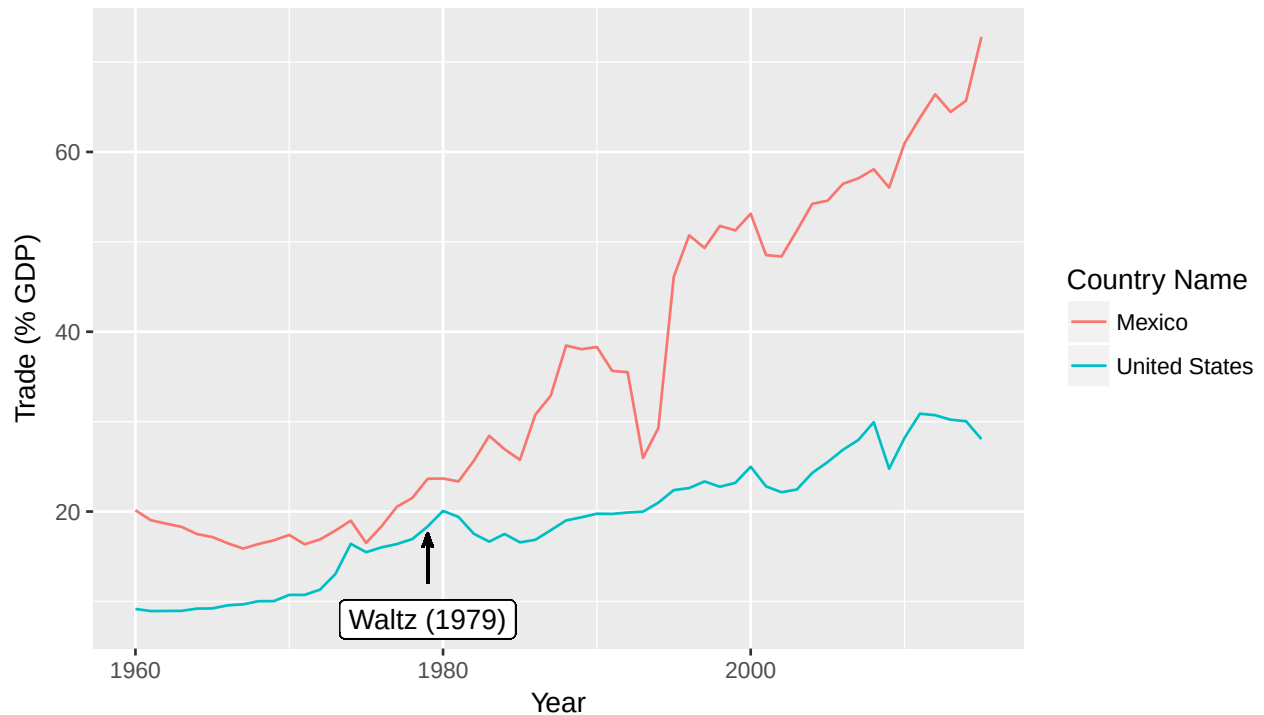
Notice there are 15,048 rows, or 264*57. Many of the Trade column observations are missing, which you can establish with the `table(is.na())` functions.

```
longWDI <- WDI %>% select(`Country Name`, `1960`:`2016`) %>%
  gather(key = Year, value = Trade, -`Country Name`)
table(is.na(longWDI$Trade))
```

```
##
## FALSE  TRUE
## 10343  4705
```

Now that we have gathered the data, we can easily pass it to our favorite plotting technology, ggplot. Lets take some time to pull apart this complicated function.

```
longwDI %>% filter(`Country Name` %in% c("United States", "Mexico")) %>%
  na.omit() %>% ggplot(aes(as.numeric(Year), as.numeric(Trade))) +
  geom_line(aes(group = `Country Name`, color = `Country Name`)) +
  labs(x = "Year", y = "Trade (% GDP)") + geom_label(aes(x = 1979,
y = 8, label = "Waltz (1979)")) + geom_segment(aes(x = 1979,
xend = 1979, y = 12, yend = 17.5), arrow = arrow(angle = 20,
length = unit(0.2, "cm")))
```



- We begin by selecting the US and Mexico. Note the use of the logical `%in%` and the use of the quote around Country Name, a problem typical of variable names with spaces in them.
- We then pipe the filtered result to a `na.omit()` command, which drops any row with any variable that is missing. This is a rough way to deal with data.
- The data frame is then fed to a ggplot, where we put Year and Trade on the x and y axis respectively. `as.numeric` transforms our variables, which are currently not numeric, to numeric types.
- The last 6 lines add features to the plot, including a nice little arrow, which is a kind of line segment.

Spread (the Reverse Gather)

Having gathered the data, we can also spread it to do interesting analysis. Here we spread the long data to have Countries as columns, and each years trade as an observation.

```
longWDI %>% filter(`Country Name` %in% c("United States", "Mexico",  
      "Canada")) %>% spread(`Country Name`, Trade)
```

```
## # A tibble: 57 × 4  
##   Year      Canada      Mexico `United States`  
## *   <chr>      <chr>      <chr>      <chr>  
## 1  1960 35.10161493 20.1442244  9.166206516  
## 2  1961 35.80468206 19.03587177  8.929522457  
## 3  1962 35.57546192 18.65200205  8.940670963  
## 4  1963 35.70254656 18.29925888  8.957093642  
## 5  1964 37.34214148 17.49509066  9.200933217  
## 6  1965 37.18922883 17.16500598  9.224149523  
## 7  1966 38.64337337 16.47319507  9.570552147  
## 8  1967 39.52099659 15.87936757  9.678542416  
## 9  1968 41.17989796 16.38955219  10.0265252  
## 10 1969 42.3883808 16.79897033  10.04020002  
## # ... with 47 more rows
```

Calculating statistics by groups of data using group_by.

This week we will study a dataset of all police shootings in 2015. I split up the URL so that it can fit on the page, but you may use the full URL directly in your code.

```
URLchunkA <- "https://raw.githubusercontent.com/washingtonpost/"
URLchunkB <- "data-police-shootings/master/fatal-police-shootings-data.csv"
shootings <- read_csv(paste0(URLchunkA, URLchunkB))
```

```
## Parsed with column specification:
## cols(
##   id = col_integer(),
##   name = col_character(),
##   date = col_date(format = ""),
##   manner_of_death = col_character(),
##   armed = col_character(),
##   age = col_integer(),
##   gender = col_character(),
##   race = col_character(),
##   city = col_character(),
##   state = col_character(),
##   signs_of_mental_illness = col_character(),
##   threat_level = col_character(),
##   flee = col_character(),
##   body_camera = col_character()
## )
```

We have already seen some of dplyr's power, particularly in its ability to quickly do subgroup analysis, here we are calculating the mean of some variables by state.

```
shootings %>% group_by(state) %>% summarise(meanage = mean(age,
  na.rm = TRUE), prop_white = mean(race == "W", na.rm = TRUE))
```

```
## # A tibble: 51 × 3
##   state meanage prop_white
##   <chr>   <dbl>     <dbl>
## 1    AK 34.63636 0.5000000
## 2    AL 39.09756 0.7000000
## 3    AR 42.20000 0.8125000
## 4    AZ 34.83146 0.5402299
## 5    CA 34.82609 0.3171521
## 6    CO 35.47458 0.5090909
## 7    CT 35.00000 0.8333333
## 8    DC 36.88889 0.0000000
## 9    DE 35.00000 0.7500000
## 10   FL 35.96610 0.4695652
## # ... with 41 more rows
```

We can easily arrange the outcomes with the arrange function. Here we use desc to make the states sort by descending order.

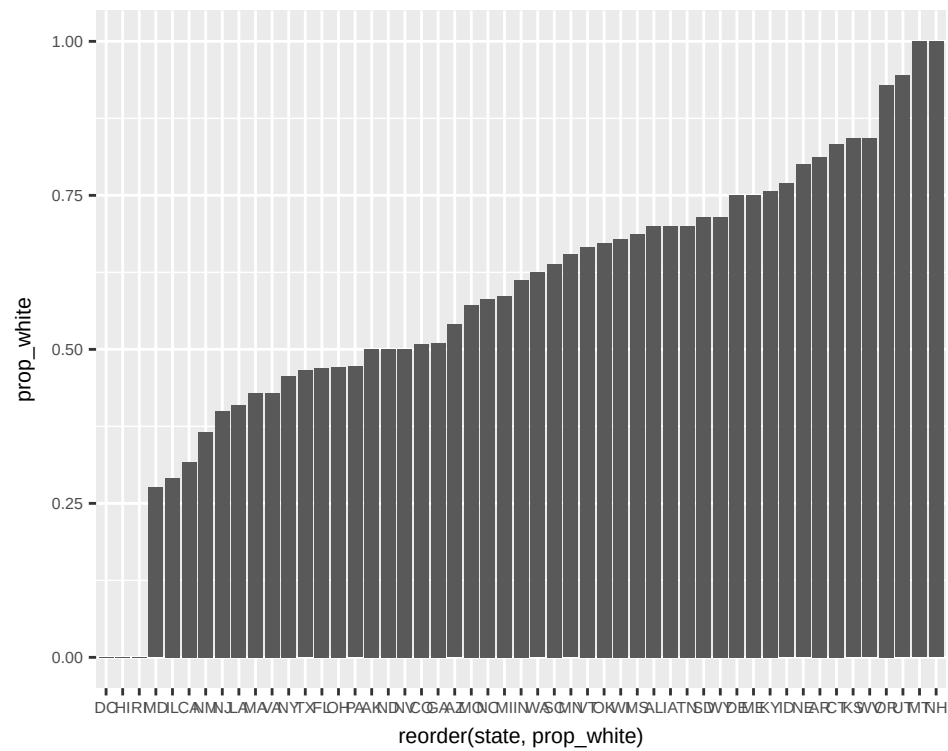
```
shootingbystate <- shootings %>% group_by(state) %>% summarise(meanage = mean(age,
  na.rm = TRUE), prop_white = mean(race == "W", na.rm = TRUE),
  n_killed = length(race)) %>% arrange(desc(prop_white))
```

```
shootingbystate
```

```
## # A tibble: 51 × 4
##   state meanage prop_white n_killed
##   <chr>   <dbl>     <dbl>   <int>
## 1    MT 35.33333 1.0000000     9
## 2    NH 32.60000 1.0000000     5
## 3    UT 36.11111 0.9444444    18
## 4    OR 41.13333 0.9285714    30
## 5    KS 37.00000 0.8421053    19
## 6    WV 45.50000 0.8421053    23
## 7    CT 35.00000 0.8333333     6
## 8    AR 42.20000 0.8125000    20
## 9    NE 36.00000 0.8000000    15
## 10   ID 43.00000 0.7692308    13
## # ... with 41 more rows
```

We can order within bar plots by using the `reorder` command:

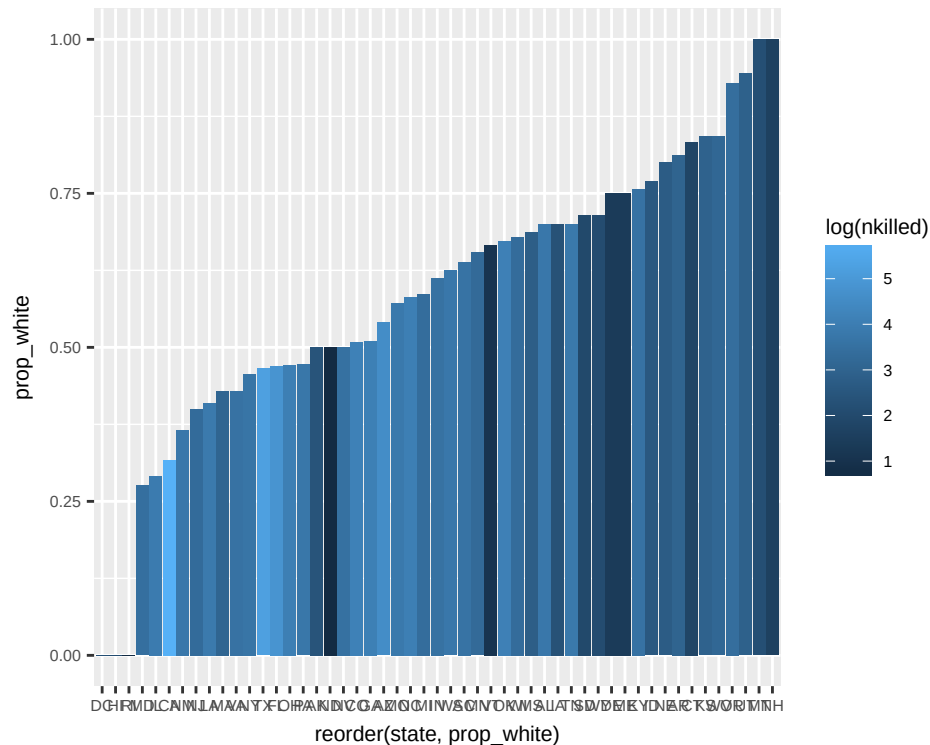
```
ggplot(shootingbystate, aes(reorder(state, prop_white), prop_white)) +
  geom_bar(stat = "identity") + theme(text = element_text(size = 7.5))
```



```
# Here I change the text size
```

We can add a fill aesthetic for the proportion of those killed that are white as follows:

```
ggplot(shootingbystate, aes(reorder(state, prop_white), prop_white,
  fill = log(nkilled))) + geom_bar(stat = "identity") + theme(text = element_text(size = 7.5))
```



We might think that the number killed should be normalized somehow, perhaps by population. We have the most accurate, 2010 numbers, in the package, noncensus:

```
library(noncensus)
data(states)
```

We have talked about help files, but one thing to begin to become good at is finding the underlying structure of R things: We can do that with the str() command

```
str(states)
```

```
## 'data.frame': 56 obs. of 7 variables:
## $ state : Factor w/ 56 levels "AK","AL","AR",...: 1 2 3 5 6 7 8 9 10 11 ...
## $ name : chr "Alaska" "Alabama" "Arkansas" "Arizona" ...
## $ region : Factor w/ 4 levels "Midwest","Northeast",...: 4 3 3 4 4 4 2 3 3 3 ...
## $ division : Factor w/ 9 levels "East North Central",...: 9 5 7 8 9 8 4 6 6 6 ...
## $ capital : chr "Juneau" "Montgomery" "Little Rock" "Phoenix" ...
## $ area : chr "589757" "51609" "53104" "113909" ...
## $ population: chr "4779736" "710231" "6392017" "2915918" ...
```

```
shootingbystatecensus <- shootingbystate %>% left_join(states) %>%
  mutate(nkilledpercap = nkilled/as.numeric(population))
```

```
## Joining, by = "state"
```

```
## Warning in left_join_impl(x, y, by$x, by$y, suffix$x, suffix$y): joining
## factor and character vector, coercing into character vector
```

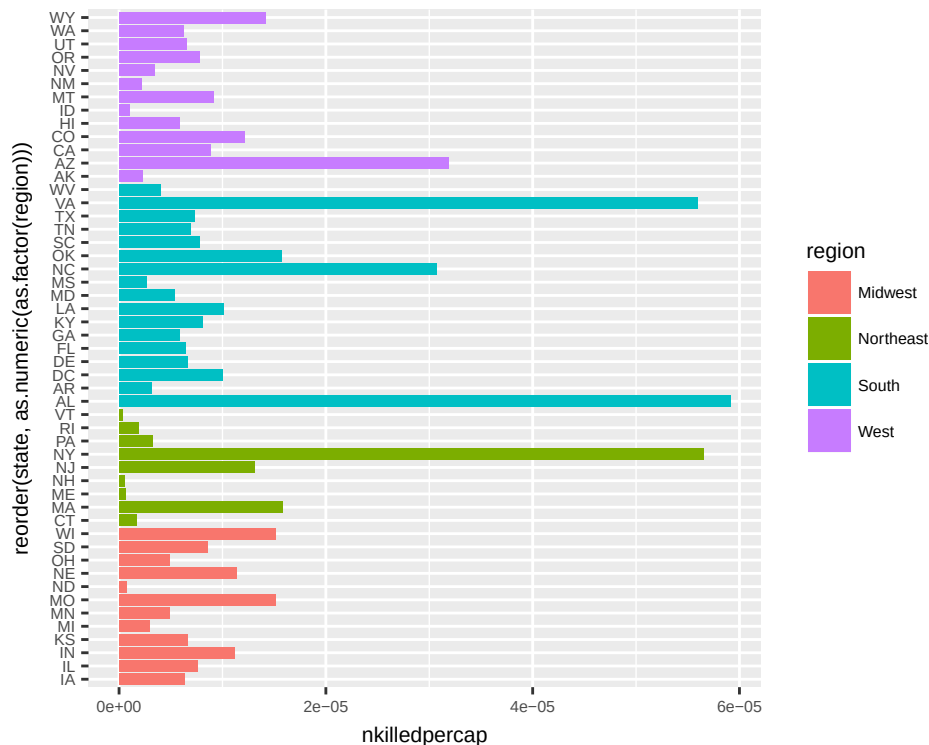

Here is a trick that uses factors and numbers. Region is normally a string, but if you turn it into a factor, it will make the variable categorical. Factors are somewhat challenging to deal with, but they can be useful. If you turn region into a factor, it will in fact turn region into an alphabetically ordered list of numbered categories.

For example, consider the following string vector:

```
stringstates <- c("Ohio", "Michigan", "California")
as.numeric(stringstates)
## Warning: NAs introduced by coercion
## [1] NA NA NA
as.numeric(factor(stringstates))
## [1] 3 2 1
```

We then use those numbered categories to reorder the states.

```
ggplot(shootingbystatecensus, aes(reorder(state, as.numeric(as.factor(region))),
  nkilledpercap, fill = region)) + geom_bar(stat = "identity") +
  coord_flip() + theme(text = element_text(size = 7.5))
```



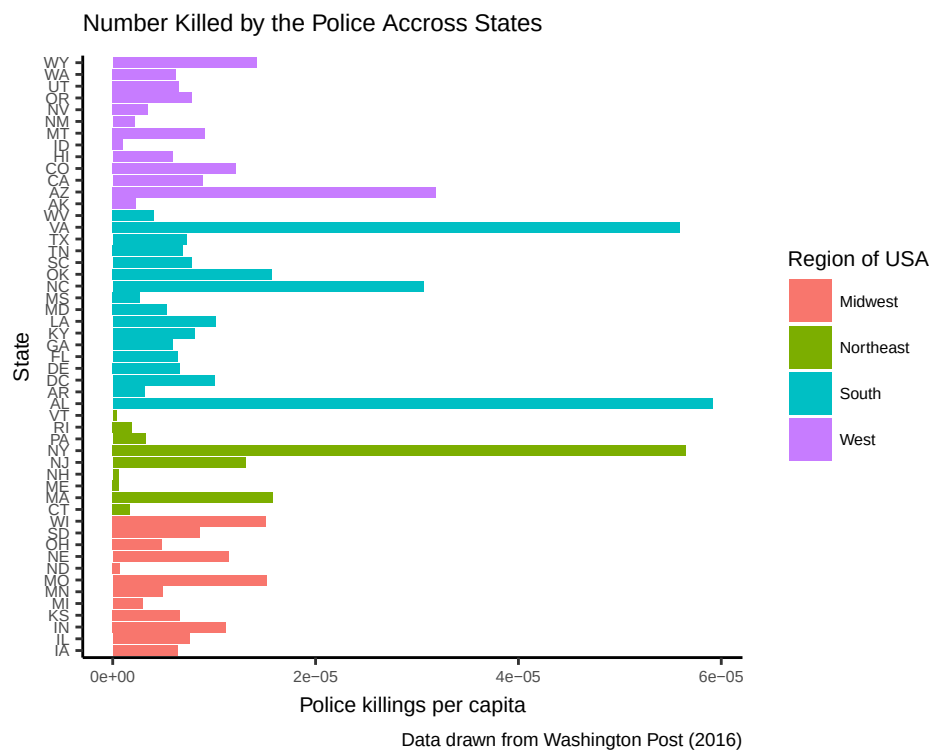
NOTE: If you want to deal with the date format, check out library(lubridate)

Labeling the plot

The labeling technology in ggplot2 has improved a great deal. In the following code, we relabel every aspect of the plot.

```
shootingplot <- ggplot(shootingbystatecensus, aes(reorder(state,
  as.numeric(as.factor(region))), nkilledpercap, fill = region)) +
  geom_bar(stat = "identity")

shootingplot + labs(title = "Number Killed by the Police Accross States",
  x = "State", y = "Police killings per capita", fill = "Region of USA",
  caption = "Data drawn from Washington Post (2016)") + coord_flip() +
  theme_classic() + theme(text = element_text(size = 7.5))
```



Cleaning data with tidyr, government data, and the VA.

```
library(jsonlite)
library(tidyr)
```

So far, all of the data we have been using is in a nice *tidy* format. That is, all of the data we have been using has each row characterise an observed outcome, here a police shooting, and each column is a characteristic of that shooting. Much of the time, we need to transform data before it can be used.

Here are a bunch of variables from the government on Veterans Administration services, a major provider of mental health care. We are again getting this directly from the internet. Again, we are only using paste so it fits on the pdf in a nice way.

```
VAest <- fromJSON(paste0("https://raw.githubusercontent.com/vacobrydsk/",
  "VHA-Files/master/NEPEC_AnnualDataSheet_MH_FY15.json"), flatten = T)
```

```
head(VAest)
```

```
##                                     Category
## 1 MH Service Utilization With a Possible or Confirmed Mental Illness
## 2 MH Service Utilization With a Possible or Confirmed Mental Illness
## 3 MH Service Utilization With a Possible or Confirmed Mental Illness
## 4 MH Service Utilization With a Possible or Confirmed Mental Illness
## 5 MH Service Utilization With a Possible or Confirmed Mental Illness
## 6 MH Service Utilization With a Possible or Confirmed Mental Illness
##                                     Item
## 1 Proportion of Veterans with Possible Mental Illness Seen in Any Mental Health
## 2 Proportion of Veterans with Possible Mental Illness Seen in Any Mental Health
## 3 Proportion of Veterans with Possible Mental Illness Seen in Any Mental Health
## 4 Proportion of Veterans with Possible Mental Illness Seen in Any Mental Health
## 5 Proportion of Veterans with Possible Mental Illness Seen in Any Mental Health
## 6 Proportion of Veterans with Possible Mental Illness Seen in Any Mental Health
##   Station Value ValueType
## 1      402 60.8%   Percent
## 2      405 54.6%   Percent
## 3      518 78.1%   Percent
## 4      523 72.7%   Percent
## 5      608 58.1%   Percent
## 6      631 74.4%   Percent
```

This is another classic example of ‘untidy’ data. Notice that the variables are listed in columns, alongside the observation name (here a ‘Station’). *tidy data* is data that has exactly one observation per row, with columns for each variable.

The `tidyr` package handles this sort of thing. The `spread` command takes as inputs the variables, here “item”, and the inputs to that variable, here “Value” and then creates a new column for each.

```
VAtidy <- VAest %>% select(-ValueType, -Category) %>% spread(Item,
  Value)
```

Unfortunately, the variable names are pretty long. We can get the content of a long column name by using quotes like this: `VAtidy$'Proportion of Non-Mental Health Services Rendered to VHA Users with a Confirmed Mental Illness - Outpatient Encounters'`

But it is better to fix the column names.

Renaming variables

There are 41 variables in the dataset. Ideally, each variable name is descriptive, not too long, and has no spaces in it. To do so, it is often easiest to do things by hand. Here I used the `dput` command to print the names in a nice concatenated list. Then it was just a matter of picking 41 names.

```
# dput(names(VAtidy))

fixednames <- c("Station", "ADC_all_beds", "ADC_inpatient", "ADC_residential",
  "ADC_psych_encounters", "ADC_psych_unique", "Avestops_MH",
  "Avestops_Psychiatry", "Avestops_Substance", "Setting_Inpatient",
  "Setting_Outpatient", "Setting_Resident", "N_serviceusers",
  "N_serviceusers_confirmed", "N_serviceusers_possible", "Pct_confirmed",
  "Pct_possible", "NMH_Prop_community_confirmed", "NMH_Prop_inpatient_confirmed",
  "NMH_Prop_outpatient_confirmed", "NMH_Prop_community_possible",
  "NMH_Prop_inpatient_possible", "NMH_Prop_outpatient_possible",
  "Prop_Vets_confirmed_any", "Prop_Vets_confirmed_outpatient",
  "Prop_Vets_confirmed_inpatient", "Prop_Vets_confirmed_resident",
  "Prop_Vets_possible_any", "Prop_Vets_possible_outpatient",
  "Prop_Vets_possible_inpatient", "Prop_Vets_possible_resident",
  "testing_encounters", "testing_uniques", "ServiceUserswithMH",
  "Total_NHM_stays_community", "Total_NHM_stays_inpatient",
  "Total_NHM_stays_outpatient", "Total_Service_Users", "Unique_Vets_inpatient",
  "Unique_Vets_outpatient", "Unique_Vets_resident")

names(VAtidy) <- fixednames
```

Strings to Numbers

VAtidy data has many of its columns as strings rather than numbers. We can turn these into numbers by doing the following, which deploys regular expressions, which we will learn about next week. For now, just know that in the pattern part of `str_replace_all`, we are using a code which reads, replace either a percent sign or a comma. The pipe `|` is the logical or.

```
stringtonumber <- function(sortanumber) {  
  as.numeric(str_replace_all(as.character(sortanumber), "%|,",  
    ""))  
}
```

```
VAtidy2 <- VAtidy %>% mutate_all(stringtonumber)
```

```
## Warning in `__dplyr_colwise_fun`(c("358", "402", "405", "436", "437",  
## "438", : NAs introduced by coercion
```

To connect this data to the shootings, here are the locations of each VA station:

```
VAfacilities <- fromJSON(paste0("https://raw.githubusercontent.com/department-of-veterans-affairs",  
  "/VHA-Facilities/master/VAFacilityLocation.json"), flatten = T)
```

```
VAfacdta <- VAfacilities$VAFacilityData %>% mutate(Sttn = facility_id)
```